

Data Module for BG-Forecasting

This module provides comprehensive data loading, preprocessing, and PyTorch integration for blood glucose forecasting research. The implementation is **based on and adapted from the GluPred project**, with significant enhancements for research workflows and modern deep learning practices.

Overview

The data module follows a clean separation of concerns:

- **Loaders:** Extract raw data from XML files without modifications
- **Preprocessors:** Handle all data transformations, feature engineering, and quality control
 - Enhanced basal insulin handling with proper temporal logic
 - Debug information to track preprocessing steps
- **PyTorch Datasets:** Convert preprocessed data into training-ready tensors using window-based approach

Module Structure

```
benchmark/data/  
├── __init__.py          # Module initialization  
├── loaders.py           # Raw data loading from XML files  
├── preprocessors.py     # Data preprocessing and feature engineering  
├── torch_dataset.py     # PyTorch dataset classes for training  
├── test_data.py        # Test pipeline with complete examples  
└── README.md           # This documentation
```

Data Flow Pipeline

```
Raw OhioT1DM XML Files → Loader → Preprocessor → PyTorch Dataset → Model  
Training
```

1. **Loading:** XML files are parsed and combined into raw DataFrames
2. **Preprocessing:**
 - Temporal event handling (basal insulin, bolus, etc.)
 - Missing values handling with configurable strategies
 - Feature engineering with cyclical time encoding
 - CSV saving for processed data inspection
3. **Dataset Creation:** Time series sequences are extracted using window approach
4. **Model Training:** PyTorch DataLoaders feed data to neural networks

Supported Datasets

OhioT1DM Dataset

The primary supported dataset is the OhioT1DM (Ohio Type 1 Diabetes Mellitus) dataset from the Ohio University Diabetes Research Center.

Versions Supported:

- **2018:** 6 patients, training/test splits
- **2020:** 6 patients, training/test splits

Data Types:

- Continuous Glucose Monitoring (CGM)
 - Insulin delivery (basal and bolus)
 - Enhanced basal rate handling with proper temporal logic
 - Automatic conversion from hourly to sampling-interval rates
 - Temporary basal rate handling with duration support
 - Meal information (carbohydrates, meal type)
 - Physiological data (heart rate, galvanic skin response, skin temperature)
 - Lifestyle data (sleep, work, exercise)
-

Testing and Validation (`test_data.py`)

The module includes comprehensive test examples to validate the entire data pipeline and demonstrate proper usage patterns.

Test Functions

`example_single_patient_training()`

Purpose: Demonstrates complete pipeline for training on individual patients.

What it tests:

- Data loading from XML files
- Preprocessing pipeline execution
- PyTorch dataset creation
- DataLoader configuration
- End-to-end data flow validation

Output: Validates data shapes, columns, and creates training-ready DataLoaders.

`example_multi_patient_training()`

Purpose: Tests multi-patient scenarios for transfer learning and population models.

What it tests:

- Multiple patient data loading

- Cross-patient data consistency
- Transfer learning dataset preparation
- Global vs. target patient data separation

Output: Creates datasets for global training and target patient fine-tuning.

Time Series Forecasting Approach

The module implements a window-based approach for blood glucose forecasting, which is the standard methodology in the field:

Window-Based Sequence Creation

- **Input Window:** Fixed-length sequence of past observations (default: 12 samples = 60 minutes)
- **Prediction Horizon:** Number of future time steps to predict (configurable)
- **Sliding Window:** Sequences are created with a sliding window to maximize training data

Advantages of Window Approach

1. **Clinical Relevance:** Matches physiological dynamics of glucose metabolism
2. **Model Compatibility:** Ideally suited for recurrent models (LSTM, GRU) and transformers
3. **Flexibility:** Supports both single-step and multi-horizon predictions
4. **Data Efficiency:** Generates multiple training examples from continuous time series

Implementation

The windowing approach is implemented in both:

- `preprocessors.py`: `create_sequences()` method creates NumPy arrays
- `torch_dataset.py`: `OhioDataset` class creates PyTorch-ready tensors with proper validation

Running Tests

```
# From the benchmark/data directory
cd benchmark/data
python test_data.py
```

Expected Output:

```
=====
SINGLE PATIENT PYTORCH DATASET EXAMPLE
=====
Loading data for patient 540...
Train data shape: (X, Y)
Test data shape: (X, Y)
✓ Single patient dataset ready for training!
=====
```

MULTI-PATIENT PYTORCH DATASET EXAMPLE

```
=====
Loading data for patients [540, 544, 552, 567, 584, 596]...
Global dataset (all other patients): X sequences
Target patient train: Y sequences
Target patient test: Z sequences
✓ Multi-patient datasets ready for transfer learning!
```

```
=====
ALL EXAMPLES COMPLETED!
```

```
=====
DATA PIPELINE TESTS PASSED -- ready for model training!
```

Test Validation Points

1. **Data Loading:** Verifies XML parsing and DataFrame creation
2. **Preprocessing:** Confirms feature engineering and missing value handling
3. **PyTorch Integration:** Validates tensor creation and normalization
4. **Sequence Extraction:** Tests time series windowing
5. **Multi-Patient Handling:** Ensures consistent processing across subjects

Using Tests for Development

The test functions serve multiple purposes:

- **Pipeline Validation:** Ensure changes don't break the data flow
- **Usage Examples:** Demonstrate proper API usage
- **Data Quality Checks:** Validate preprocessing results

Core Components

1. Data Loaders (`loaders.py`)

`OhioT1DMDataloader`

Purpose: Parse OhioT1DM XML files and return raw, unmodified DataFrames.

Key Features:

- Handles both 2018 and 2020 dataset versions
- Parses XML files for CGM, insulin, meals, and physiological data
- Combines multiple data sources with proper time alignment
- Returns raw data without any preprocessing

Usage:

```
from benchmark.data.loaders import OhioT1DMDataloader
```

```

loader = OhioT1DMDataloader(
    data_dir="/path/to/data",
    sampling_rate=5, # 5-minute intervals
    version="2020"
)

# Load single patient
patient_df = loader.load_patient_data(540, 'train')

# Load multiple patients
train_data = load_ohiot1dm_data(
    data_dir="/path/to/data",
    patient_ids=[540, 544, 552],
    mode='train',
    version='2020'
)

```

Columns Returned:

- **glucose**: Blood glucose levels (mg/dL)
- **basal**: Basal insulin rate (units/hour)
- **bolus**: Bolus insulin (units)
- **bolus_dur**: Bolus duration (minutes)
- **carbs**: Carbohydrate intake (grams)
- **meal_type**: Type of meal
- **hr**: Heart rate (bpm)
- **gsr**: Galvanic skin response (μ S)
- **st**: Skin temperature ($^{\circ}$ C)

2. Data Preprocessors (**preprocessors.py**)

OhioBGDataPreprocessor

Purpose: Transform raw data into ML-ready format with comprehensive preprocessing.

Key Features:

- **Missing Data Handling:** Fill missing values, handle -1 markers
- **Temporal Event Processing:** Apply insulin duration, meal effects
- **Feature Engineering:** Time-based features, glucose derivatives, cumulative features
- **Outlier Detection:** Statistical outlier identification and flagging
- **Normalization:** Multiple scaling options (standard, minmax, robust)
- **Sequence Creation:** Extract input-output pairs for forecasting

Processing Pipeline:

1. **Basic Preprocessing:** Handle missing values, temporal events
2. **Feature Engineering:** Create time-based and glucose-derived features
3. **Outlier Detection:** Flag anomalous values

4. **Normalization:** Scale features for ML models
5. **Sequence Creation:** Extract training sequences

Usage:

```
from benchmark.data.preprocessors import OhioBGDataPreprocessor

preprocessor = OhioBGDataPreprocessor(
    target_column='glucose',
    sampling_rate=5
)

# Full preprocessing pipeline
processed_df = preprocessor.preprocess_patient_data(
    raw_df,
    apply_basic_preprocessing=True,
    include_feature_engineering=True,
    normalize=True,
    handle_missing='interpolate'
)

# Create sequences for training
X, y = preprocessor.create_sequences(
    processed_df,
    sequence_length=12,      # 1 hour history
    prediction_horizon=6,    # 30 minutes ahead
    step_size=1
)
```

Feature Engineering:

- **Temporal Features:** Hour, day of week, weekend indicators
- **Glucose Features:** Differences, rolling statistics, trends
- **Cumulative Features:** Insulin and carb sums over time windows
- **Outlier Flags:** Binary indicators for anomalous values

3. PyTorch Datasets ([torch_dataset.py](#))

OhioDataset

Purpose: Convert preprocessed DataFrames into PyTorch-compatible datasets for deep learning.

Key Features:

- **Sequence Extraction:** Create time series sequences without missing values
- **Standardization:** Z-score normalization using training statistics
- **Insulin Smoothing:** Redistribute extended bolus over time (adapted from GluPred)
- **Flexible Features:** Support univariate (glucose-only) or multivariate models
- **Consistent Normalization:** Share statistics between train/test sets

Usage:

```
from benchmark.data.torch_dataset import OhioDataset,
prepare_personal_data
from torch.utils.data import DataLoader

# Create datasets
train_dataset = OhioDataset(
    raw_df=preprocessed_train_df,
    sequence_length=12,
    prediction_horizon=1,
    unimodal=False # Use multiple features
)

test_dataset = OhioDataset(
    raw_df=preprocessed_test_df,
    sequence_length=12,
    prediction_horizon=1,
    external_mean=train_dataset.mean,
    external_std=train_dataset.std,
    unimodal=False
)

# Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# Training loop
for sequences, targets in train_loader:
    # sequences: (batch_size, sequence_length, n_features)
    # targets: (batch_size, prediction_horizon)
    predictions = model(sequences)
    loss = criterion(predictions, targets)
    # ... training code
```

GluPred Integration

This implementation is **based on and inspired by the GluPred project**, with significant adaptations:

From GluPred:

- **Dataset structure:** Core PyTorch Dataset class design
- **Insulin smoothing:** Algorithm for distributing extended bolus over time
- **Sequence extraction:** Method for finding contiguous valid sequences
- **Normalization approach:** Z-score standardization using training statistics

Enhancements for BG-Forecasting:

- **Modular architecture:** Separated loading, preprocessing, and dataset creation

- **Advanced preprocessing:** Comprehensive feature engineering and outlier detection
- **Flexible data handling:** Support for missing values and various data quality issues
- **Research workflow:** Designed for experimental reproducibility

Key Algorithm: Insulin Smoothing

The insulin smoothing algorithm (adapted from GluPred) handles extended bolus delivery:

```
def _smooth_bolus(self, bolus, bolus_dur):  
    """Redistribute extended bolus over delivery duration."""  
    for i in range(len(bolus)):  
        if bolus[i] > 0 and bolus_dur[i] > 0:  
            # Find consecutive identical bolus values  
            j = 1  
            while i + j < len(bolus) and bolus[i + j] == bolus[i]:  
                j += 1  
            # Distribute total dose over time steps  
            bolus[i:i+j] = bolus[i:i+j] / j  
    return bolus
```

This ensures that extended insulin delivery (e.g., for high-fat meals) is represented accurately for ML models.

Usage Examples

Quick Start with Tests

The fastest way to get started is to run the test pipeline:

```
# Clone and navigate to data module  
cd benchmark/data  
  
# Run comprehensive pipeline tests  
python test_data.py  
  
# This will validate:  
# ✓ Data loading from XML files  
# ✓ Preprocessing pipeline  
# ✓ PyTorch dataset creation  
# ✓ Multi-patient handling  
# ✓ Transfer learning setup
```

Research Features

Experimental Reproducibility

- **Deterministic processing:** Consistent results across runs
- **Configurable parameters:** Easy experiment configuration
- **Detailed logging:** Track preprocessing steps and statistics
- **Version control:** Dataset version tracking

Extensibility

The architecture supports:

- **Additional datasets:** Easy integration of new data sources
- **Custom preprocessing:** Modular preprocessing steps
- **Different ML frameworks:** Not limited to PyTorch
- **Various prediction tasks:** Single/multi-step, single/multi-variate

Requirements

Python Dependencies

- `pandas >= 1.3.0`
- `numpy >= 1.20.0`
- `scikit-learn >= 1.0.0`
- `torch >= 1.9.0` (for PyTorch datasets)
- `datetime`, `xml.etree.ElementTree` (standard library)

Data Requirements

- OhioT1DM dataset XML files in proper directory structure
- Sufficient disk space for processed data
- Memory requirements depend on number of patients and sequence length

Acknowledgments

This implementation is based on the excellent work from the **GluPred project**:

GluPred: A Glucose Prediction System using Deep Learning

Original authors: [Cui, Ran and Hettiarachchi, Chirath and Nolan, Christopher J and Daskalaki, Elena and Suominen, Hanna]

Our implementation adapts their PyTorch dataset design and insulin smoothing algorithms while extending functionality for comprehensive research workflows.

Citation

If you use this data module in your research, please cite both:

1. **This BG-forecasting project**
2. **The original GluPred project** that inspired the core dataset design

Troubleshooting

Common Issues

1. **Import Errors:** Ensure all dependencies are installed
2. **Missing Data Files:** Check data directory structure
3. **Memory Issues:** Reduce batch size or sequence length
4. **NaN Values:** Check preprocessing configuration

Getting Help

- Check the comprehensive examples in the parent directory
- Review error messages for specific issues
- Ensure data files are in the expected format and location

Future Enhancements

- **Additional datasets:** Support for more diabetes datasets
- **Real-time processing:** Streaming data capabilities
- **Performance optimization:** Faster preprocessing and loading

Debug Features

The module now includes comprehensive debug information to help understand the preprocessing pipeline:

Basal Insulin Processing Debug

The `_apply_basal_rates()` method in `preprocessors.py` provides detailed logging:

- Number of basal rate changes detected
- Conversion from hourly rates to interval rates
- Temporary basal event details (start, end, affected rows)
- Statistical summaries of processed values

How to Use Debug Output

Debug prints are integrated into the code and appear during preprocessing:

```
[BASIC] Starting basic preprocessing on X rows
[TEMPORAL] Processing basal insulin rates...
[BASAL] Found N non-missing basal values out of X rows
[BASAL] After forward fill: X valid basal values
[BASAL] Found N temporary basal events to process
[BASAL] Temp event: X.XX U/hr → X.XXXX U/5min
...
```

To save debugging output to a file, redirect stdout:

```
import sys
# Redirect stdout to file
original_stdout = sys.stdout
with open('preprocessing_debug.log', 'w') as f:
    sys.stdout = f
    # Run preprocessing
    preprocessed_data = preprocessor.basic_preprocessing(df)
    sys.stdout = original_stdout
```

This module provides a robust foundation for blood glucose forecasting research, built on proven algorithms from GluPred and enhanced for modern deep learning workflows with improved physiological fidelity.